TALLER DE DISEÑO DE SOFTWARE 1

Programación Web — Paso a Paso

GUÍA DE TRABAJOS PRÁCTICOS

N° 07

Manejo del DOM

eventos — módulos — vectores de objetos

Año 2026

Prof. Ing. Daniel Maldonado

Objetivos de la guía

Al finalizar esta guía, el alumno debe poder:

- Capturar elementos de la pantalla con querySelector y delegar eventos con onclick.
- Leer lo que ingresa el usuario (value y checked) y mostrar resultados con textContent.
- Escribir funciones como arrow function en un archivo aparte (modelo.js), exportarlas e importarlas desde el controlador.
- Combinar inputs, select y checkbox para resolver problemas con condiciones.
- Recorrer un vector de objetos literales con forEach, find y filter, contando y acumulando.

⚠ Importante

Cada ejercicio se resuelve con tres archivos: vista.html (HTML y CSS a gusto), controlador.js y modelo.js. El HTML carga el controlador con type="module".

Todas las funciones van en modelo.js, como arrow function y exportadas. Las funciones calculan y devuelven: nunca usan console.log ni tocan la pantalla. Eso lo hace el controlador.

Cada ejercicio lleva un botón Limpiar que deja inputs y resultados en blanco.

1 Capturar, leer y mostrar

De qué se trata este bloque
Capturar los elementos de la pantalla, leer lo que escribe el usuario, calcular con una función y mostrar el resultado.

✏️ Ejercicio 51 — Bienvenida

Ingresar apellido y nombre, y mostrar en pantalla un saludo de bienvenida con ambos en mayúsculas.

✏️ Ejercicio 52 — IVA

Ingresar el importe de una factura y mostrar cuánto es el IVA (21%).

✏️ Ejercicio 53 — Comidas rápidas

Realizar una aplicación web para tomar el pedido de un local de comidas rápidas.

- Elegir una comida de una lista: hamburguesa $7.800, lomito $10.500, pollo frito $7.000.
- Indicar la cantidad.
- Indicar si lleva papas y cuántas porciones (cada porción cuesta $3.500).
- Mostrar el total a pagar.

2 Checkbox y cálculos

De qué se trata este bloque
Aparece un elemento nuevo: el checkbox. No se lee con value sino con checked, que vale true o false.

✏️ Ejercicio 54 — Compra online

Ingresar el subtotal de una compra. Si tiene cupón, se descuenta el 10% del subtotal. Si pide envío a domicilio, se suman $4.500. Mostrar el total.

✏️ Ejercicio 55 — Factura

Ingresar el importe base de una factura. Según corresponda, se le suma IVA (21%) y percepción de Ingresos Brutos (3%). Mostrar el detalle y el total.

✏️ Ejercicio 56 — Dividir la cuenta

Ingresar el total de una cuenta, la cantidad de personas y el porcentaje de propina. Mostrar cuánto paga cada uno.

3 Más lógica en las funciones

De qué se trata este bloque
Combinar select, inputs y checkbox, con condiciones que dependen unas de otras.

✏️ Ejercicio 57 — Cine

Elegir el tipo de entrada (general $9.000, estudiante $6.500, jubilado $5.000), la cantidad y si es en 3D (+$2.000 por entrada). Si es miércoles, el total tiene 20% de descuento. Mostrar el total.

✏️ Ejercicio 58 — Estacionamiento

Elegir el vehículo (moto $800 por hora, auto $1.500, camioneta $2.000) e ingresar las horas. Si se queda más de 5 horas, el total tiene 15% de descuento. Si usa cochera cubierta, se le suma 25% al total ya descontado. Mostrar el total.

✏️ Ejercicio 59 — Recibo de sueldo

Ingresar el sueldo bruto. Se descuenta jubilación (11%), obra social (3%) y PAMI (3%). Si está afiliado al sindicato, se descuenta además un 2%. Mostrar cada descuento y el sueldo neto.

✏️ Ejercicio 60 — Agua por bloques (desafío)

Ingresar los m³ consumidos. Los primeros 50 m³ se cobran a $350 cada uno (si consumió menos, igual paga 50). Del m³ 51 al 70 se cobran a $555,20, y lo que pase de 70 a $1.552,20. Mostrar el importe a pagar.

Para verificar: 30 m³ → $17.500 · 55 m³ → $20.276 · 85 m³ → $51.887.

4 Vectores de objetos

De qué se trata este bloque
Los datos viven en modelo.js como un vector de objetos literales. Los resultados de este bloque se muestran por consola.

✏️ Ejercicio 61 — Productos

Trabajar con el siguiente vector de productos. Un botón para cada consigna:

Copiá este vector en tu modelo.js:

```
export const productos = [
  { codigoBarras: 7790001000011, descripcion: "ALFAJOR DE CHOCOLATE",
    rubro: "GOLOSINAS", precio: 1200, stock: 40 },
  { codigoBarras: 7790001000028, descripcion: "GASEOSA COLA 1.5L",
    rubro: "BEBIDAS", precio: 3200, stock: 15 },
  { codigoBarras: 7790001000035, descripcion: "LECHE ENTERA 1L",
    rubro: "LACTEOS", precio: 2400, stock: 3 },
  { codigoBarras: 7790001000042, descripcion: "YERBA MATE 1KG",
    rubro: "ALMACEN", precio: 6800, stock: 12 },
  { codigoBarras: 7790001000059, descripcion: "YOGUR BEBIBLE 1L",
    rubro: "LACTEOS", precio: 2800, stock: 9 },
  { codigoBarras: 7790001000066, descripcion: "FIDEOS TALLARIN 500G",
    rubro: "ALMACEN", precio: 1900, stock: 5 },
  { codigoBarras: 7790001000073, descripcion: "QUESO CREMOSO 1KG",
    rubro: "LACTEOS", precio: 9500, stock: 4 },
  { codigoBarras: 7790001000080, descripcion: "AGUA MINERAL 500ML",
    rubro: "BEBIDAS", precio: 1500, stock: 60 }
];
```

- Listar todos los productos (forEach).
- Mostrar el producto de un código de barras ingresado (find).
- Mostrar los productos con precio entre dos valores ingresados (filter).
- Contar cuántos productos tienen precio entre dos valores ingresados (forEach con contador).
- Contar cuántos productos pertenecen a un rubro elegido en un select (forEach con contador).
- Calcular el valor total del inventario: la suma de precio × stock de todos los productos (forEach con acumulador).
- Mostrar el producto más caro (forEach, recorriendo y comparando).

✏️ Ejercicio 62 — Alumnos

Trabajar con el siguiente vector de alumnos. Un botón para cada consigna:

Copiá este vector en tu modelo.js:

```
export const alumnos = [
  { legajo: 1001, apellido: "GOMEZ", nota: 9, carrera: "PROGRAMACION" },
  { legajo: 1002, apellido: "RODRIGUEZ", nota: 4, carrera: "DISENO WEB" },
  { legajo: 1003, apellido: "FERNANDEZ", nota: 7, carrera: "PROGRAMACION" },
  { legajo: 1004, apellido: "LOPEZ", nota: 10, carrera: "REDES" },
  { legajo: 1005, apellido: "MARTINEZ", nota: 5, carrera: "DISENO WEB" },
  { legajo: 1006, apellido: "SANCHEZ", nota: 6, carrera: "REDES" }
];
```

- Listar todos los alumnos (forEach).
- Mostrar el alumno de un legajo ingresado (find).
- Mostrar los aprobados, con nota de 6 o más (filter).
- Contar cuántos alumnos aprobaron (forEach con contador).
- Calcular el promedio general de notas (forEach con acumulador y contador).
- Mostrar el alumno con la nota más alta (forEach, recorriendo y comparando).
- Contar cuántos alumnos hay en una carrera elegida en un select (forEach con contador).

✏️ Ejercicio 63 — Celulares

Trabajar con el siguiente vector de celulares:

Copiá este vector en tu modelo.js:

```
export const celulares = [
  { id: 1, marca: "SAMSUNG", modelo: "A15", precio: 280000, stock: 10 },
  { id: 2, marca: "SAMSUNG", modelo: "A35", precio: 420000, stock: 0 },
  { id: 3, marca: "MOTOROLA", modelo: "G54", precio: 310000, stock: 6 },
  { id: 4, marca: "MOTOROLA", modelo: "G84", precio: 450000, stock: 3 },
  { id: 5, marca: "XIAOMI", modelo: "REDMI 13", precio: 260000, stock: 0 },
  { id: 6, marca: "XIAOMI", modelo: "REDMI NOTE 13", precio: 380000, stock: 8 }
];
```

- Listar todos los celulares (forEach).
- Mostrar el celular de un id ingresado (find).
- Filtrar por marca (select con la opción "Todas"), por precio máximo y por un checkbox "Solo con stock". Los tres filtros son opcionales y se pueden combinar (filter).

✏️ Ejercicio 64 — Empleados

Trabajar con el siguiente vector de empleados:

Copiá este vector en tu modelo.js:

```
export const empleados = [
  { legajo: 201, apellido: "PEREZ", sector: "SISTEMAS", sueldo: 1800000 },
  { legajo: 202, apellido: "GOMEZ", sector: "VENTAS", sueldo: 1200000 },
  { legajo: 203, apellido: "DIAZ", sector: "SISTEMAS", sueldo: 2100000 },
  { legajo: 204, apellido: "RUIZ", sector: "ADMINISTRACION", sueldo: 1100000 },
  { legajo: 205, apellido: "LOPEZ", sector: "VENTAS", sueldo: 1350000 },
  { legajo: 206, apellido: "SOSA", sector: "ADMINISTRACION", sueldo: 1250000 }
];
```

Elegir un sector en un select y mostrar:

- Los empleados de ese sector (filter).
- La cantidad de empleados y el total de sueldos del sector (forEach con acumulador).
- El empleado de mayor sueldo del sector (forEach, recorriendo y comparando).